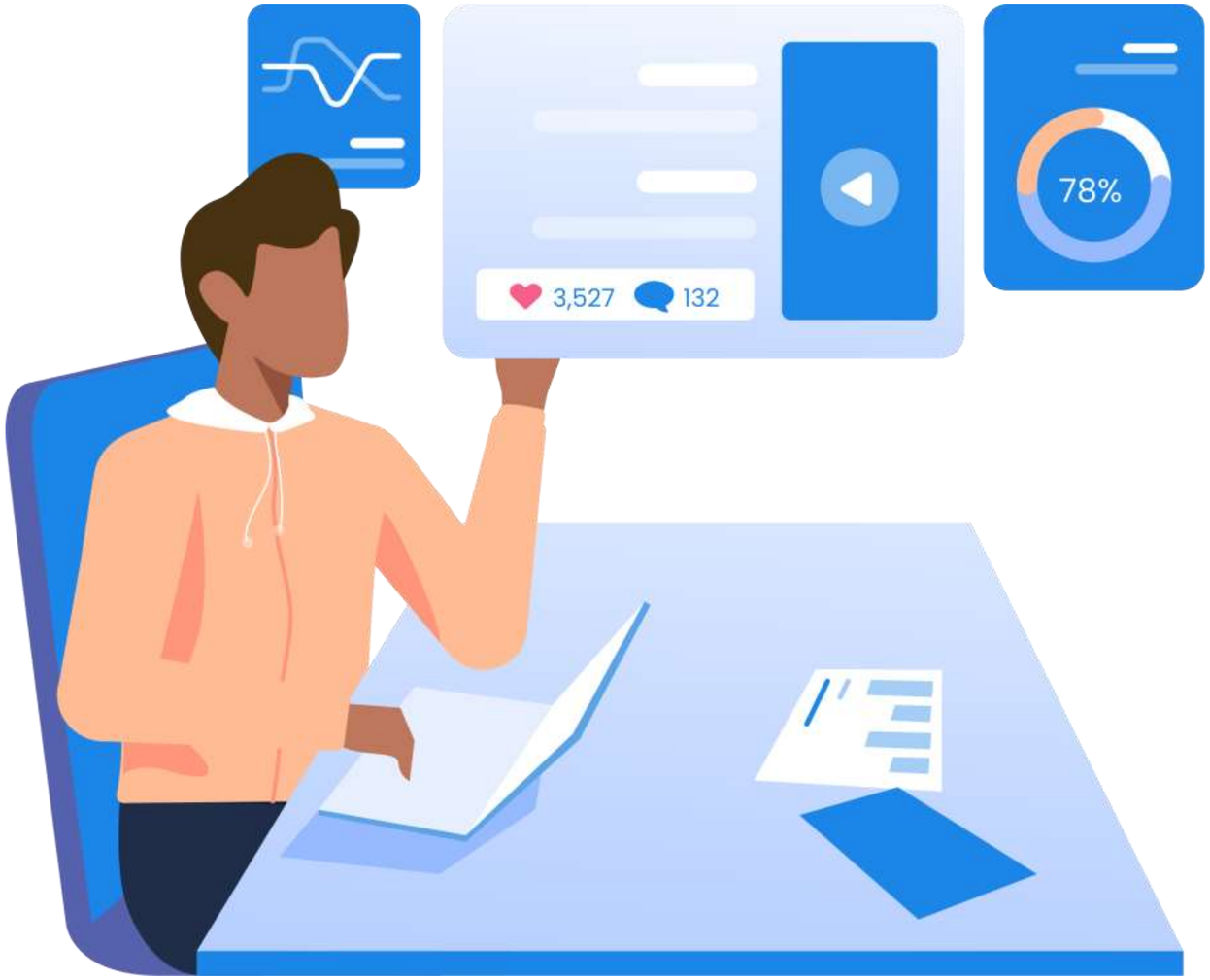
Flower Image Classification with Deep Learning

Problem and Objective

問題：如何有效且準確地分類花朵圖片，並在大量影像資料中保持良好的運算效率與模型穩定度。

目標：

- 建立能自動辨識花朵種類的深度學習模型
- 透過高效的資料載入方式提升訓練流程
- 使用資料增強與 Dropout 改善過度擬合問題
- 提升模型在測試資料上的準確度與泛化能力
- 將訓練完成的模型轉換為 TensorFlow Lite，以便部署於行動裝置與 IoT 設備



Import the Library and Download the Images

在開始模型訓練之前，我們先匯入 TensorFlow 以及相關的程式庫，用來處理影像、建立模型、與視覺化資料。

```
import matplotlib.pyplot as plt
import numpy as np
import PIL
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
```

- matplotlib.pyplot（plt）：用來顯示圖片或圖表，讓我們可以看到資料的樣子。
- numpy（np）：用來快速處理大量數字，就像功能更強的計算器。PIL：用來讀取、開啟或處理圖片檔案。
- TensorFlow（tf）：用來建立和訓練 AI 模型，是整個專案的核心工具。
- keras / layers / Sequential：TensorFlow 裡的模組，用來更方便地建立神經網路模型。

在這一步，我們先下載花朵圖片的資料集，然後查看圖片的數量、大小、分類方式等基本資訊。**這樣可以幫助我們了解資料是否完整，並確定 AI 模型需要處理什麼樣的圖片。**

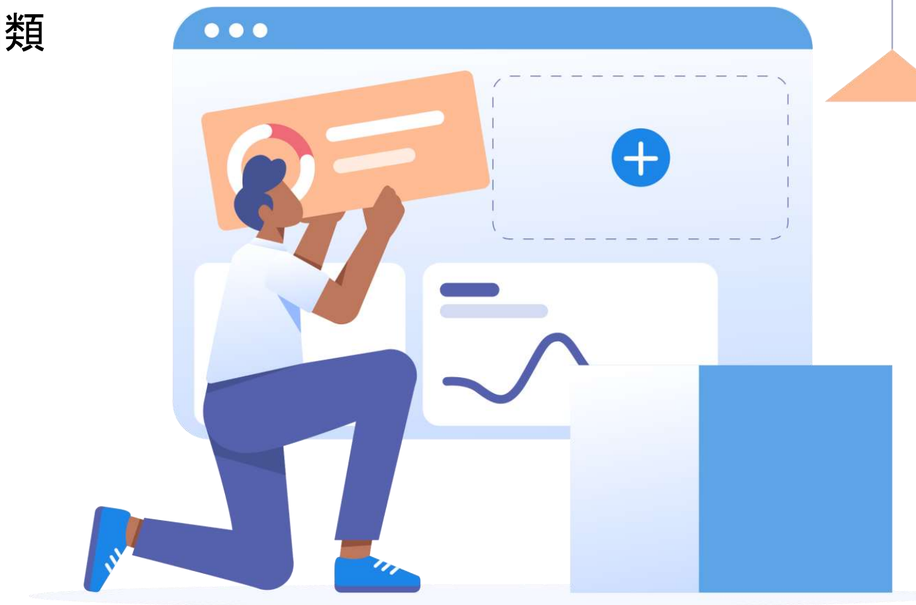
```
flower_photo/
  daisy/
  dandelion/
  roses/
  sunflowers/
  tulips/
```

```
import pathlib

dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"
data_dir = tf.keras.utils.get_file("flower_photos.tgz", origin=dataset_url, extract=True)
data_dir = pathlib.Path(data_dir).parent / "flower_photos_extracted" / "flower_photos"

Downloading data from https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz
228813984/228813984 ━━━━━━━━━━━━━━━━━━━━ 11s 0us/step
```

資料集是由已經標記好的圖片所組成，用來讓 AI 學習並了解這些相關的事物

下載完成後，我們一共收到了 3,670 張圖片，作為這份資料集的副本，分成 5 個類別：雛菊、蒲公英、玫瑰、向日葵和鬱金香。

```
image_count = len(list(data_dir.glob('*/*.jpg')))
print(image_count)

3670
```



Convert this images to keras dataset

在這一步，我們使用 Keras 提供的 image_dataset_from_directory 工具來載入圖片資料。首先先設定圖片的大小與 batch 數量，之後從資料夾中自動讀取所有圖片，並依照資料夾的名稱把圖片分成不同的類別。同時，我們把 80% 的圖片當作訓練資料，剩下的 20% 作為驗證資料，用來檢查模型是否過度擬合。載入完成後，系統也會顯示每個資料集中圖片的數量，以及所有類別的名稱，方便我們了解資料的組成情況。

在載入並整理好資料之後，下一步是將訓練資料視覺化。我們先展示訓練資料集中的前九張圖片，讓我們更清楚地了解資料的內容與每個類別的大致特徵。

```
batch_size = 32
img_height = 180
img_width = 180
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(
  data_dir,
  validation_split=0.2,
  subset="training",
  seed=123,
  image_size=(img_height, img_width),
  batch_size=batch_size)
```

```
Found 3670 files belonging to 5 classes.
Using 2936 files for training.
```

```
val_ds = tf.keras.utils.image_dataset_from_directory(
  data_dir,
  validation_split=0.2,
  subset="validation",
  seed=123,
  image_size=(img_height, img_width),
  batch_size=batch_size)
```

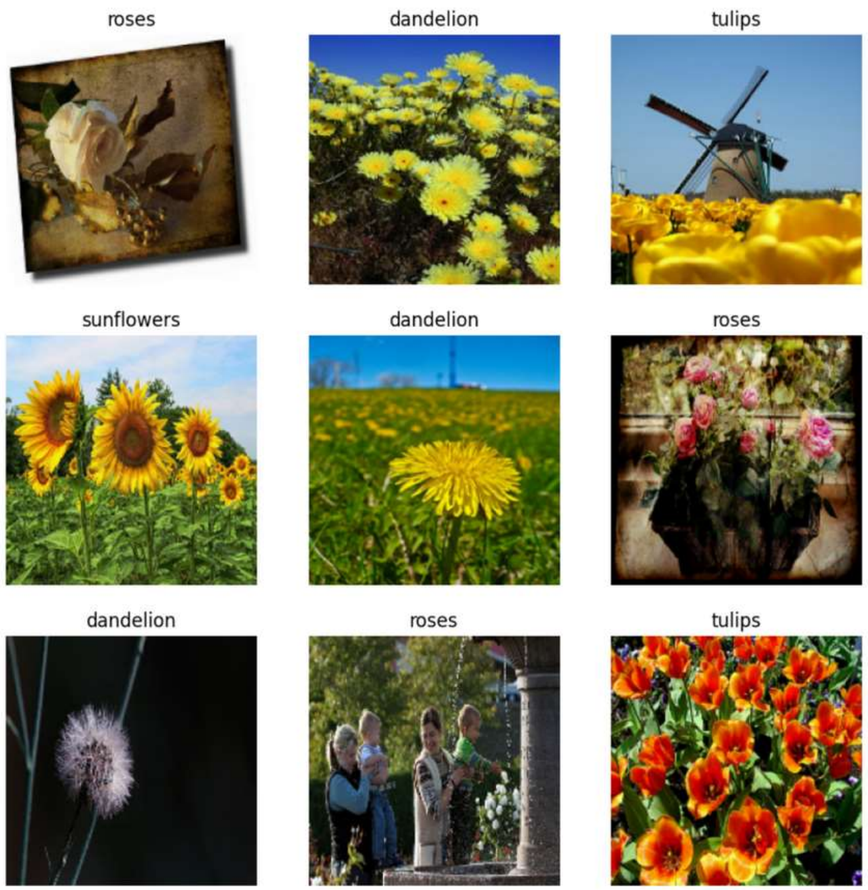
```
Found 3670 files belonging to 5 classes.
Using 734 files for validation.
```

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
  for i in range(9):
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(images[i].numpy().astype("uint8"))
    plt.title(class_names[labels[i]])
    plt.axis("off")
```

在視覺化完訓練資料後，我們接著查看資料集在實際訓練時的輸入格式。這段程式碼示範如何從 train_ds 中取出一個 batch，並列出圖片與標籤的形狀，確認資料是否以正確的大小與格式載入。這個步驟能讓我們了解模型在 model.fit() 中會收到什麼樣的資料，也確保後續的訓練可以順利進行。

```python
for image_batch, labels_batch in train_ds:
  print(image_batch.shape)
  print(labels_batch.shape)
  break
```

```
(32, 180, 180, 3)
(32,)
```



Preprocessing and standardize

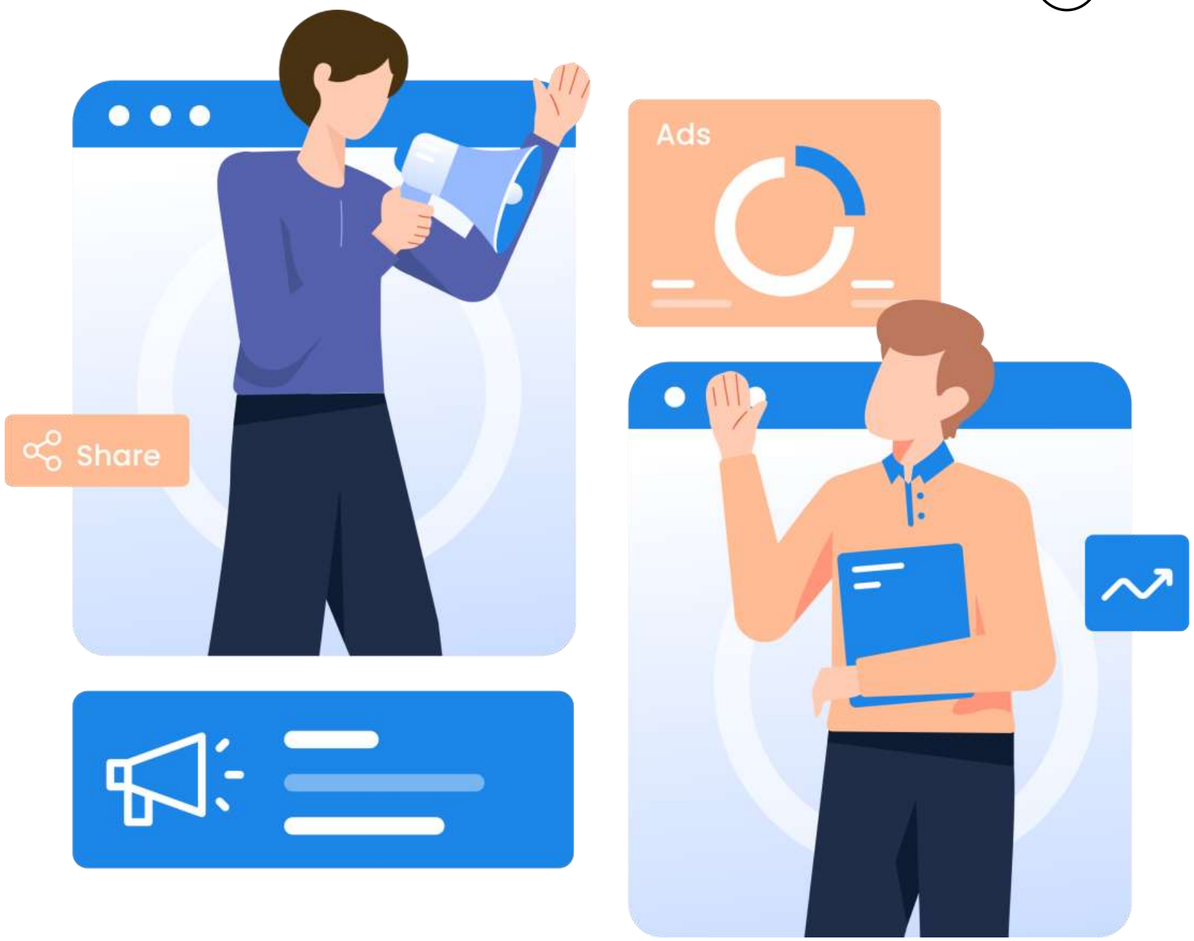
在確認資料格式正確之後，下一步是提升資料載入的效能。這裡我們使用 cache() 和 prefetch() 來優化資料集，讓模型在訓練時可以更快地取得圖片，而不會因為讀取速度太慢而卡住。cache() 會把處理過的資料暫存在記憶體中，而 prefetch() 則能讓系統在模型訓練的同時提前準備下一批資料。這些設定能大幅改善訓練速度與整體效能。

```
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

我們把圖片的像素值從 0-255 縮放到 0-1，讓資料更適合神經網路訓練並提升模型的穩定性。

```
normalization_layer = layers.Rescaling(1./255)
normalized_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
image_batch, labels_batch = next(iter(normalized_ds))
first_image = image_batch[0]
# Notice the pixel values are now in `[0,1]`.
print(np.min(first_image), np.max(first_image))
```

```
0.0 1.0
```



Create and Compile the Model

在完成資料標準化後，我們開始建立深度學習模型。這裡使用 Keras 的 Sequential 模型，並加入多層卷積層（Conv2D）、池化層（MaxPooling2D）以及全連接層（Dense）。卷積層能從圖片中提取特徵，池化層負責縮小特徵圖、減少計算量，而最後的全連接層則用來進行分類，判斷圖片屬於五種花朵中的哪一類。

```
num_classes = len(class_names)

model = Sequential([
  layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
  layers.Conv2D(16, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(32, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(64, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Flatten(),
  layers.Dense(128, activation='relu'),
  layers.Dense(num_classes)
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/preprocessi
  super().__init__(**kwargs)
```

我們使用 Adam 優化器，並選擇 SparseCategoricalCrossentropy 作為損失函數。接著加入 accuracy 作為模型的評估指標，讓我們可以看到訓練和驗證的準確率。

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

Summary and Train the Model

在編譯完成後，我們使用 model.summary() 查看整個模型的結構，包括每一層的名稱、輸出形狀以及參數數量. 然後我們使用 model.fit() 來訓練模型，這裡設定訓練 10 個 epochs，並同時加入驗證資料（validation data），用來觀察模型在訓練過程中的學習狀況與準確率變化。

```
epochs=10
history = model.fit(
  train_ds,
  validation_data=val_ds,
  epochs=epochs
)
```

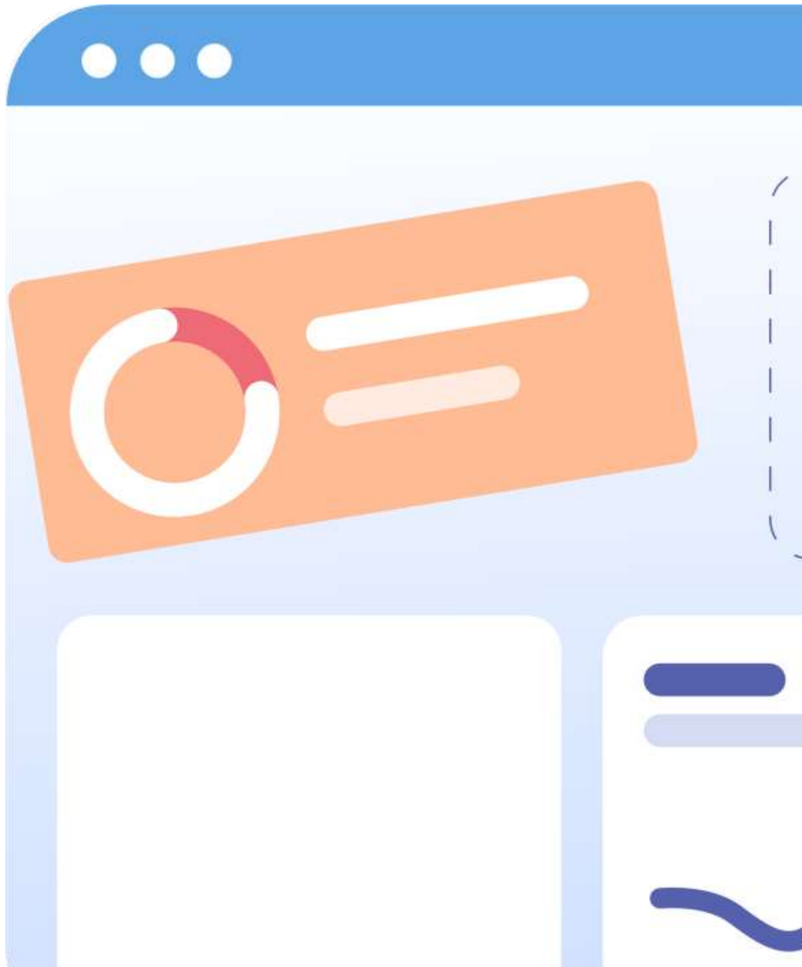
```
Epoch 1/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 15s 82ms/step - accuracy: 0.4178 - loss: 1.3694 - val_accuracy: 0.5272 - val_loss: 1.1064
Epoch 2/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.6022 - loss: 1.0180 - val_accuracy: 0.6376 - val_loss: 0.9309
Epoch 3/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.6998 - loss: 0.7887 - val_accuracy: 0.6390 - val_loss: 0.9003
Epoch 4/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.7719 - loss: 0.6507 - val_accuracy: 0.6662 - val_loss: 0.8845
Epoch 5/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.8397 - loss: 0.4481 - val_accuracy: 0.6431 - val_loss: 1.0044
Epoch 6/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.9130 - loss: 0.2728 - val_accuracy: 0.6826 - val_loss: 1.0524
Epoch 7/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.9638 - loss: 0.1212 - val_accuracy: 0.6853 - val_loss: 1.2235
Epoch 8/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.9763 - loss: 0.0721 - val_accuracy: 0.6294 - val_loss: 1.6390
Epoch 9/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.9781 - loss: 0.0875 - val_accuracy: 0.6785 - val_loss: 1.5091
Epoch 10/10
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.9948 - loss: 0.0212 - val_accuracy: 0.6730 - val_loss: 1.5928
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 180, 180, 3)	0
conv2d (Conv2D)	(None, 180, 180, 16)	448
max_pooling2d (MaxPooling2D)	(None, 90, 90, 16)	0
conv2d_1 (Conv2D)	(None, 90, 90, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 45, 45, 32)	0
conv2d_2 (Conv2D)	(None, 45, 45, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 22, 22, 64)	0
flatten (Flatten)	(None, 30976)	0
dense (Dense)	(None, 128)	3,965,056
dense_1 (Dense)	(None, 5)	645

Total params: 3,989,285 (15.22 MB)
Trainable params: 3,989,285 (15.22 MB)
Non-trainable params: 0 (0.00 B)



Visualize Training Result

在模型訓練完成後，我們把訓練過程中的 accuracy 和 loss 畫成圖表，用來觀察模型的學習情況。
從圖中可以看到：訓練與驗證的差距很大，模型可能有過擬合，且驗證準確率只有約 60%。
接下來需要進一步調整模型來提升表現。

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

Data Augmentation

我們利用 資料增強 來減少過擬合。透過圖片的隨機 翻轉、旋轉、縮放，產生更多變化的訓練資料，讓模型看見更豐富的樣本，提高泛化能力。程式示範如何把同一張圖片做多次增強，並把結果視覺化。

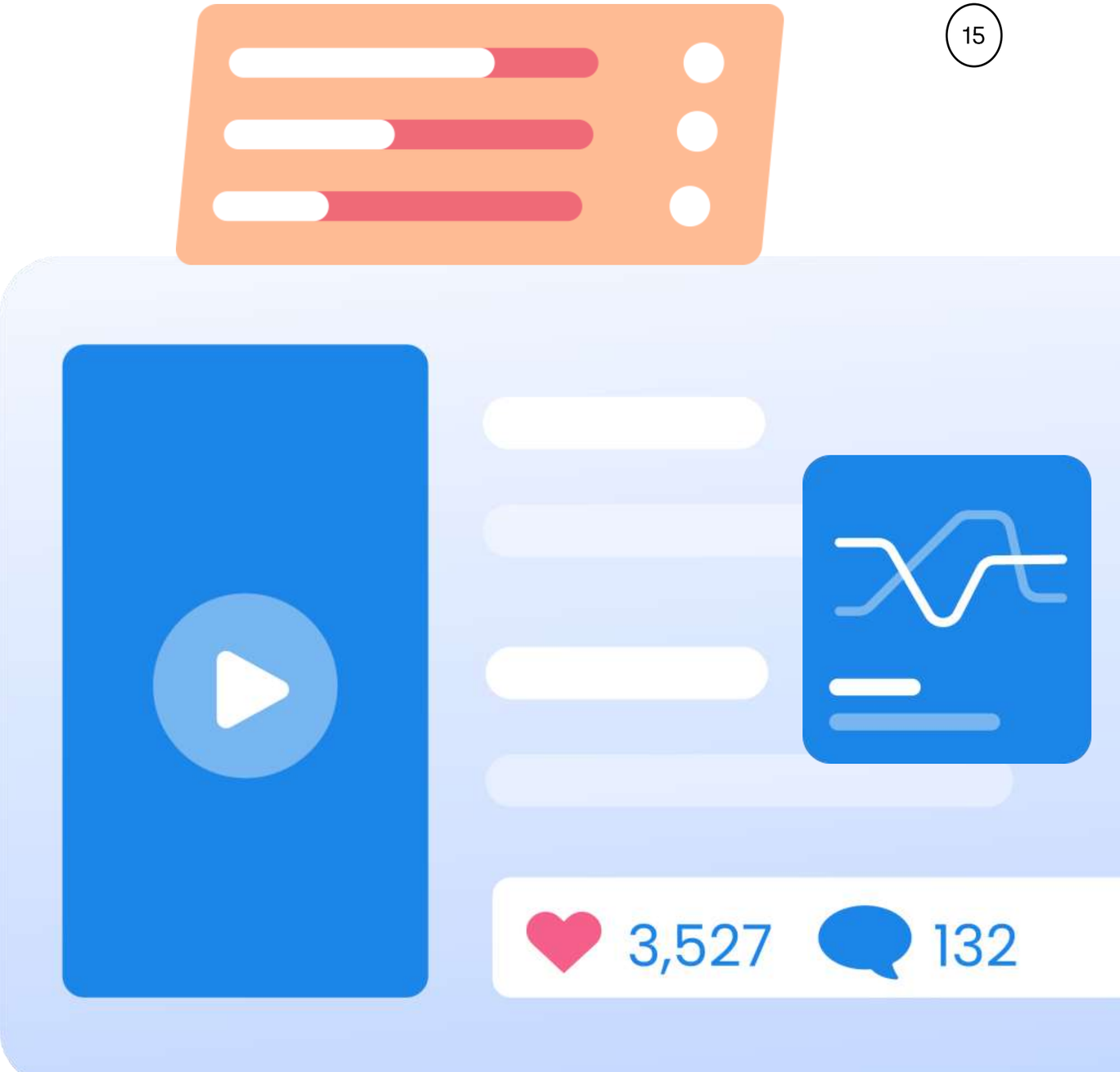
```
data_augmentation = keras.Sequential(
  [
    layers.RandomFlip("horizontal",
                      input_shape=(img_height,
                                   img_width,
                                   3)),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
  ]
)
```

```
plt.figure(figsize=(10, 10))
for images, _ in train_ds.take(1):
  for i in range(9):
    augmented_images = data_augmentation(images)
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(augmented_images[0].numpy().astype("uint8"))
    plt.axis("off")
```

Drop Out

Dropout 是另一種減少過擬合的方法。
在訓練時，它會隨機把部分神經元關掉（例如 10%、20% 或 40%），
讓模型不要過度依賴某些特徵，並提升泛化能力。
下面的程式把 Dropout 加入到模型裡，
讓模型在使用資料增強後更能避免過擬合。
接著我們加入 Dropout 後，重新編譯並訓練模型，讓模型更穩定、減少過擬合。

```
model = Sequential([
  data_augmentation,
  layers.Rescaling(1./255),
  layers.Conv2D(16, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(32, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(64, 3, padding='same', activation='relu'),
  layers.MaxPooling2D(),
  layers.Dropout(0.2),
  layers.Flatten(),
  layers.Dense(128, activation='relu'),
  layers.Dense(num_classes, name="outputs")
])
```



```python
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

```python
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential_1 (Sequential)	(None, 180, 180, 3)	0
rescaling_2 (Rescaling)	(None, 180, 180, 3)	0
conv2d_3 (Conv2D)	(None, 180, 180, 16)	448
max_pooling2d_3 (MaxPooling2D)	(None, 90, 90, 16)	0
conv2d_4 (Conv2D)	(None, 90, 90, 32)	4,640
max_pooling2d_4 (MaxPooling2D)	(None, 45, 45, 32)	0
conv2d_5 (Conv2D)	(None, 45, 45, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 22, 22, 64)	0
dropout (Dropout)	(None, 22, 22, 64)	0
flatten_1 (Flatten)	(None, 30976)	0
dense_2 (Dense)	(None, 128)	3,965,056
outputs (Dense)	(None, 5)	645

Total params: 3,989,285 (15.22 MB)
Trainable params: 3,989,285 (15.22 MB)
Non-trainable params: 0 (0.00 B)

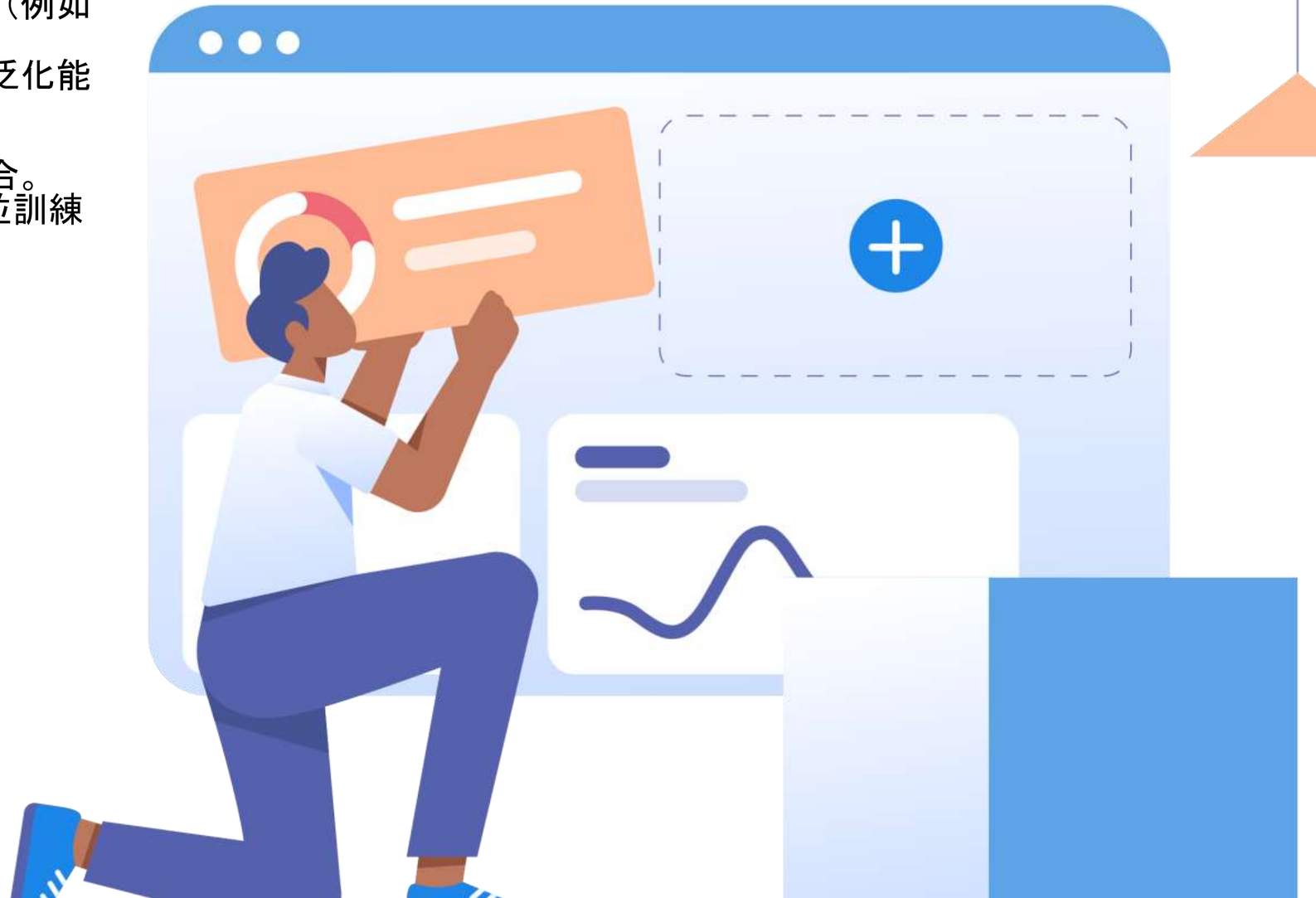
```python
epochs = 15
history = model.fit(
  train_ds,
  validation_data=val_ds,
  epochs=epochs
)
```

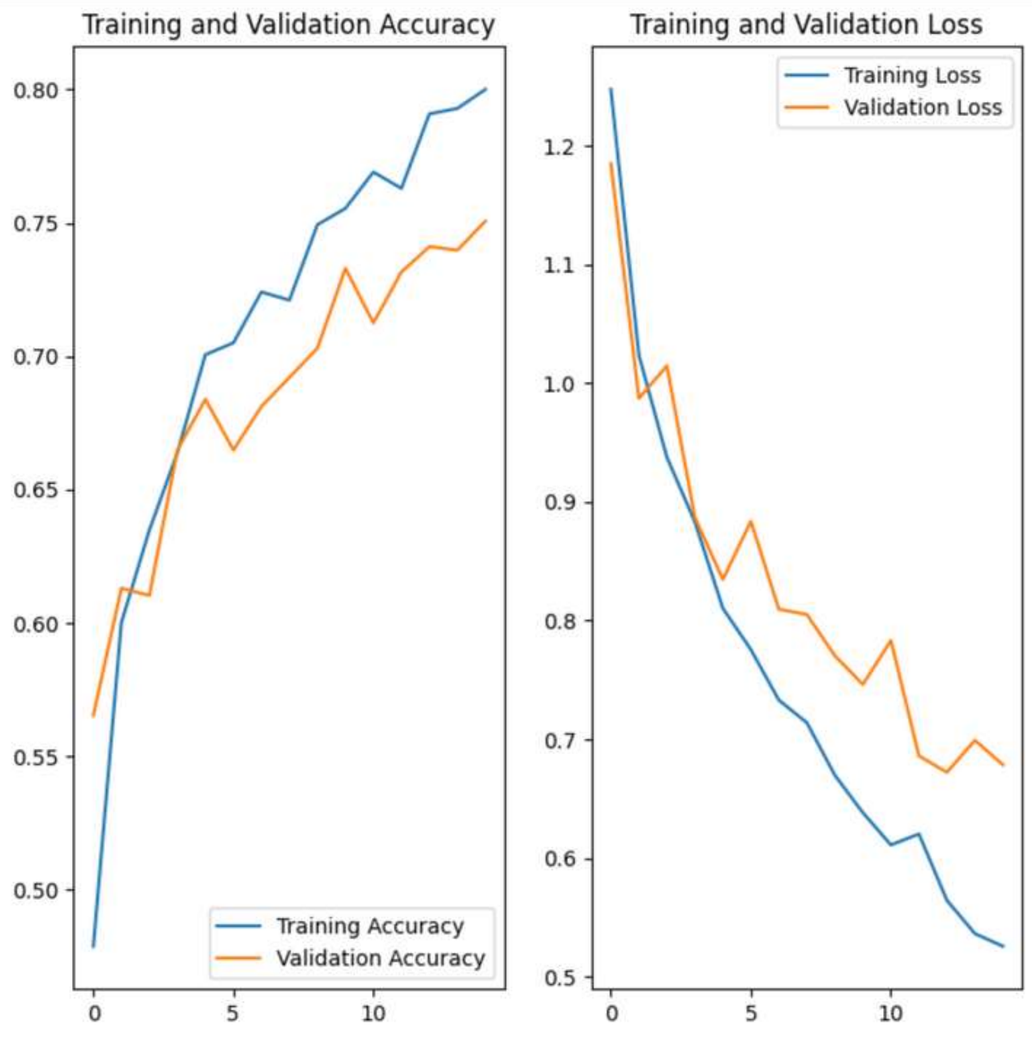
```
Epoch 1/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 7s 36ms/step - accuracy: 0.3998 - loss: 1.4213 - val_accuracy: 0.5654 - val_loss: 1.1849
Epoch 2/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 8s 28ms/step - accuracy: 0.5910 - loss: 1.0495 - val_accuracy: 0.6131 - val_loss: 0.9871
Epoch 3/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.6276 - loss: 0.9482 - val_accuracy: 0.6104 - val_loss: 1.0144
Epoch 4/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - accuracy: 0.6523 - loss: 0.9082 - val_accuracy: 0.6649 - val_loss: 0.8867
Epoch 5/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.7037 - loss: 0.7936 - val_accuracy: 0.6839 - val_loss: 0.8346
Epoch 6/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.7107 - loss: 0.7730 - val_accuracy: 0.6649 - val_loss: 0.8835
Epoch 7/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.7283 - loss: 0.7351 - val_accuracy: 0.6812 - val_loss: 0.8095
Epoch 8/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.7216 - loss: 0.6971 - val_accuracy: 0.6921 - val_loss: 0.8051
Epoch 9/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 5s 28ms/step - accuracy: 0.7516 - loss: 0.6591 - val_accuracy: 0.7030 - val_loss: 0.7703
Epoch 10/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.7478 - loss: 0.6477 - val_accuracy: 0.7330 - val_loss: 0.7462
Epoch 11/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.7695 - loss: 0.5992 - val_accuracy: 0.7125 - val_loss: 0.7832
Epoch 12/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - accuracy: 0.7604 - loss: 0.6134 - val_accuracy: 0.7316 - val_loss: 0.6860
Epoch 13/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.7927 - loss: 0.5608 - val_accuracy: 0.7411 - val_loss: 0.6722
Epoch 14/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.8024 - loss: 0.5205 - val_accuracy: 0.7398 - val_loss: 0.6991
Epoch 15/15
92/92 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.8031 - loss: 0.5281 - val_accuracy: 0.7507 - val_loss: 0.6785
```

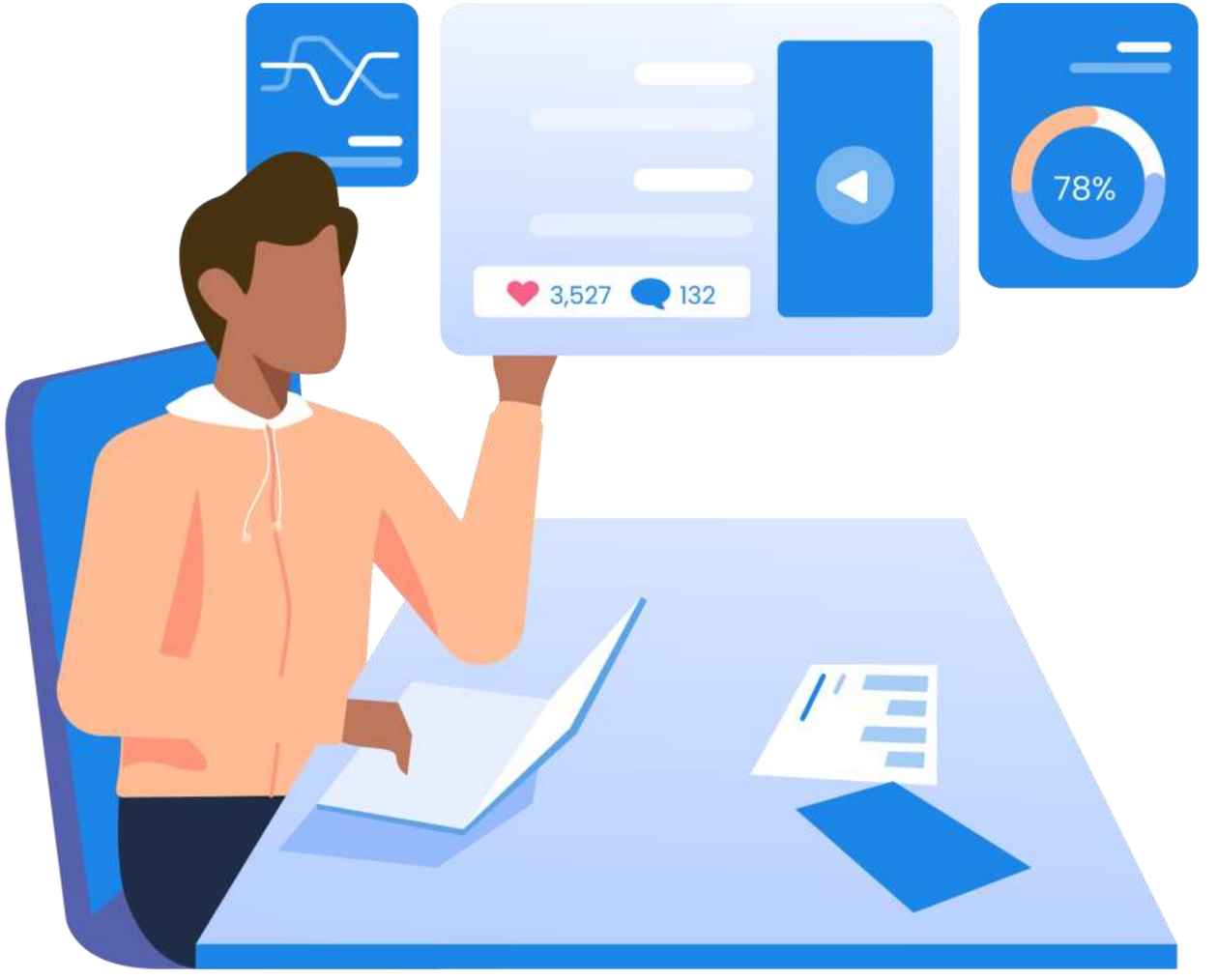
End of the Result

Dropout 是另一種減少過擬合的方法。
在訓練時，它會隨機把部分神經元關掉（例如 10%、20% 或 40%），
讓模型不要過度依賴某些特徵，並提升泛化能力。
下面的程式把 Dropout 加入到模型裡，
讓模型在使用資料增強後更能避免過擬合。
接著我們加入 Dropout 後，重新編譯並訓練模型，讓模型更穩定、減少過擬合。

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```




Training and Validation Accuracy
0.80
0.75
0.70
0.65
0.60
0.55
0.50
0
5
10
Training Accuracy
Validation Accuracy
Training and Validation Loss
1.2
1.1
1.0
0.9
0.8
0.7
0.6
0.5
0
5
10
Training Loss
Validation Loss

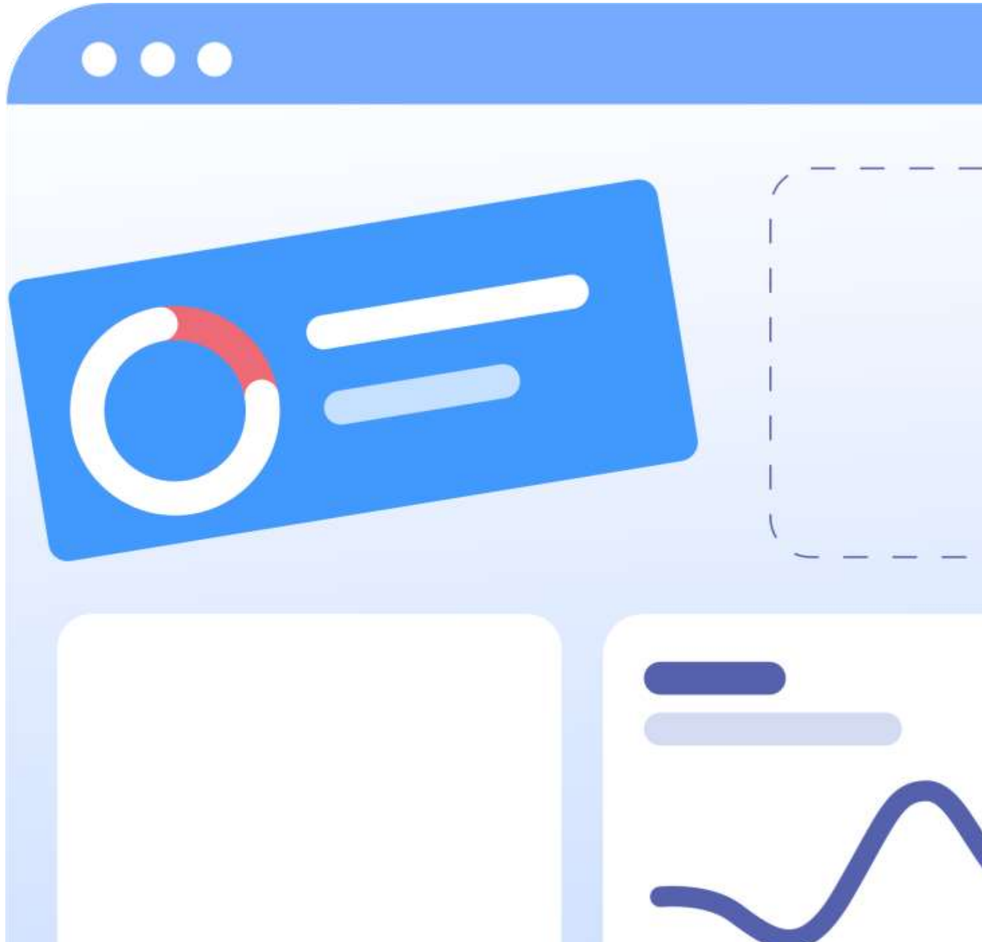

3,527
132
78%

Predict on New Data

最後，我們使用訓練好的模型去預測一張不在訓練集或驗證集中的新圖片，看看模型能否正確分類。這時候 data augmentation 和 dropout 都不會啟動，只進行純推論。

```
sunflower_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/592px-Red_sunflower.jpg"
sunflower_path = tf.keras.utils.get_file('Red_sunflower', origin=sunflower_url)

img = tf.keras.utils.load_img(
    sunflower_path, target_size=(img_height, img_width)
)
img_array = tf.keras.utils.img_to_array(img)
img_array = tf.expand_dims(img_array, 0) # Create a batch

predictions = model.predict(img_array)
score = tf.nn.softmax(predictions[0])

print(
    "This image most likely belongs to {} with a {:.2f} percent confidence."
    .format(class_names[np.argmax(score)], 100 * np.max(score))
)
```

```
Downloading data from https://storage.googleapis.com/download.tensorflow.org/example_images/592px-Red_sunflower.jpg
117948/117948 ━━━━━━━━━━━━━━━━━━━━ 0s 4us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 229ms/step
This image most likely belongs to sunflowers with a 86.24 percent confidence.
```



感謝您的聆聽